

Webots Practical Work Reflection Report

Student Name: Omar A M Issa

Teacher: Donatas Lukšys

Subject: Robotics

Submission Type: Report

Date: 4th April 2026

Webots Practical Work Reflection Report.....	1
1. Introduction	1
2. General Process of the Work.....	1
3. Why I Used C Language	2
4. Main Struggles During the Work	3
4.1 Understanding the Webots Interface	3
4.2 Understanding Nodes and Fields	4
4.3 Robot Movement Problems	4
4.4 Obstacle Avoidance Problems.....	4
4.5 Four-Wheeled Robot Problems	5
4.6 Physics Problems.....	5
4.7 Supervisor Confusion	6
5. What I Learned.....	6
6. Level I Reached.....	7
7. What I Want to Improve Next.....	8
8. Personal Reflection	8
9. Conclusion.....	9
10. References	9

1. Introduction

This report describes my practical learning process while working through the Webots tutorials. The purpose of this work was not only to finish the tutorials, but also to understand how robot simulations are created, how robot controllers work, how sensors and motors are connected to code, and how a simulation environment can be controlled and improved.

During this work, I completed the main Webots tutorials step by step, starting from basic world creation and ending with Supervisor control. The process included creating simulation worlds, adding objects, editing the scene tree, programming robot controllers in C, building a four-wheeled robot from scratch, creating a reusable PROTO robot, and using a Supervisor to control objects in the scene.

The work was challenging at different stages. Some parts were easy to follow, such as changing colors and adding simple objects. Other parts required more effort, especially when working with controllers, physics, distance sensors, wheel motors, and the Supervisor. The most useful part of the work was solving problems when the simulation did not behave as expected.

2. General Process of the Work

The work started with learning the Webots interface. At the beginning, I needed to understand where the scene tree was, how to add nodes, how to modify fields, and how to save and reset the world correctly. Tutorial 1 introduced the basic concept that a Webots world is stored as a .wbt file and that the world is made from nodes organized in a scene tree .

After that, I moved into creating objects manually. This was more difficult than simply adding ready-made objects because I had to understand the structure of a Solid node. I learned that a visible object is not enough by itself. If the object needs to collide, fall, roll, or interact physically, it also needs a correct boundingObject and Physics node. Tutorial 2 explained this by using Solid, Shape, Sphere, Box, boundingObject, and Physics nodes .

Then I worked on appearance and rendering. This part helped me understand the difference between the physical structure of an object and its visual representation. I changed colors, materials, and textures using PBRAppearance. This made the simulation clearer and more professional visually.

The next stage was controller programming. This was the most important part for me because it connected the simulation world with actual robot behavior. I used C language to write controllers, read sensor values, and control motors. Tutorial 4 introduced the controller feedback loop, where the robot continuously reads sensors, processes behavior, and sends commands to actuators .

After that, I worked with physics and compound objects. Tutorial 5 helped me understand that realistic simulation depends on mass, friction, center of mass, bounding objects, and contact properties. This was also one of the parts where I faced more problems because physics behavior is sensitive to small setup mistakes.

The most advanced modeling task was creating the four-wheeled robot from scratch. This required understanding Robot nodes, Solid nodes, HingeJoint nodes, RotationalMotor devices, and DistanceSensor devices. Tutorial 6 was important because it showed how a robot can be divided into body and wheel parts, and how each wheel can be connected using joints and controlled through motors .

After building the robot, I converted it into a reusable PROTO file. This helped me understand how robot definitions can be reused instead of copying the full structure every time. Finally, I worked with the Supervisor, which gave me a higher level of control over the simulation, such as moving objects, deleting objects, spawning new objects, tracking positions, and changing colors .

3. Why I Used C Language

I chose C language for the controllers because it is one of the main languages supported in the Webots tutorials. Webots supports several languages, including C, C++, Java, Python, and MATLAB, but C is used as a reference language in many tutorial examples. The tutorials also explain that C controllers must be compiled before they can be used, unlike Python controllers, which are interpreted .

Using C helped me understand the structure of a Webots controller more clearly. I had to include the correct Webots libraries, initialize the robot, get device references, enable sensors, write the control loop, set motor speeds, and clean up the program at the end.

The most important C functions I used were:

```
wb_robot_init();  
wb_robot_get_device();  
wb_distance_sensor_enable();  
wb_distance_sensor_get_value();
```

```
wb_motor_set_position();  
wb_motor_set_velocity();  
wb_robot_step();  
wb_robot_cleanup();
```

At first, using C made the process slower because I had to compile the code and fix errors. However, it also made the connection between the robot model and the controller very clear. For example, if the motor name in the scene tree was wheel1, the same name had to be used in the C code. If the name was wrong, the controller could not control the motor.

This helped me understand that Webots programming is not only writing code. It is also about matching the code correctly with the robot's devices in the scene tree.

4. Main Struggles During the Work

4.1 Understanding the Webots Interface

At the beginning, the Webots interface was confusing. I had difficulty knowing where to add objects, where to find nodes, and how to open or edit the fields of an object. I also had trouble understanding the difference between the 3D view and the scene tree.

Later, I understood that the 3D view only shows the simulation visually, while the scene tree is where the real structure of the simulation is edited.

Another issue was that sometimes the view became difficult to control or the code panel disappeared. This was frustrating because it made it harder to continue the work. From this, I learned that using Webots also requires knowing the interface layout, not only the programming part.

4.2 Understanding Nodes and Fields

One of the biggest early struggles was understanding nodes and fields. At first, terms like Solid, Shape, Geometry, boundingObject, Physics, DEF, and USE were confusing.

After working through the tutorials, I understood that:

- A **Solid** represents a physical object.

- A **Shape** defines how the object looks.
- A **Geometry** defines the form, such as Box, Sphere, or Cylinder.
- A **boundingObject** defines the collision shape.
- A **Physics** node makes the object dynamic.
- **DEF and USE** allow reusing the same object or geometry definition.

This was an important step because Webots depends heavily on the scene tree structure. If a node is placed in the wrong field, the object may appear visually but not behave physically.

4.3 Robot Movement Problems

In the first controller, the robot did not move the way I expected. Sometimes only the wheels moved, or the robot moved for a short time and stopped.

The reason was that I first used position control. In position control, the motor moves until it reaches a target angle. Later, I learned to use velocity control by setting the motor position to INFINITY and then setting the velocity. This allowed the robot to move continuously.

This helped me understand the difference between:

- **Position control**, where the wheel rotates to a target position.
- **Velocity control**, where the wheel keeps rotating at a given speed.

4.4 Obstacle Avoidance Problems

Obstacle avoidance was one of the most difficult parts. In Tutorial 4, the e-puck used eight distance sensors to detect obstacles. The basic controller worked, but it only made simple left and right decisions.

The main problem was that the robot could get stuck in corners. It would detect walls on one side, turn, detect another obstacle, and continue turning without escaping properly.

This showed me that simple obstacle avoidance is limited. A better controller needs more recovery behavior, such as:

- moving backward when the robot is too close to a wall,
- turning for a longer time,
- choosing a random turn direction,

- detecting when it is trapped,
- using more sensor conditions,
- or using a more advanced navigation algorithm.

This was an important learning point. A controller that works in a simple situation may fail in a more complex environment.

4.5 Four-Wheeled Robot Problems

Tutorial 6 was the hardest part because I had to build the robot from scratch. The robot had a body, four wheels, four HingeJoint nodes, four RotationalMotor devices, and two distance sensors.

The problem was that every part had to be connected correctly. If the wheel position, rotation, joint anchor, or motor name was wrong, the robot would not behave correctly. The distance sensors also had to be oriented correctly because they detect objects along their positive x-axis.

Another problem was that the robot sometimes moved forward and hit the wall. This happened because the sensor logic needed adjustment and because I did not fully understand the sensor values at first. The tutorial explains that the distance sensors in the four-wheeled robot use a measuring interval between 0 and 1000, which is different from the e-puck sensors .

This part helped me understand that robot simulation requires both mechanical setup and programming. The robot structure and the controller must work together.

4.6 Physics Problems

The physics tutorial was also challenging. In Tutorial 5, I created a dumbbell object using two spheres and one cylinder. The object needed a correct physical structure, mass, center of mass, and contact behavior.

One issue was that the dumbbell did not always behave as expected. It could slide instead of rolling, or it could fall incorrectly if the setup was not correct. The tutorial showed that when friction is removed by setting coulombFriction to 0, the dumbbell slides instead of rolling .

This helped me understand that physics in Webots is sensitive. Small changes in mass, friction, bounding objects, or object placement can strongly affect the simulation result.

4.7 Supervisor Confusion

Tutorial 8 was confusing at first because BB-8 did not move when its controller was set to `<none>`. I thought something was wrong, but then I understood that this was intended. In this tutorial, BB-8 is not controlled by its own controller. Instead, the Supervisor controls it by changing its translation field directly.

This helped me understand the difference between a normal robot controller and a Supervisor controller. A normal controller controls the robot's own sensors and actuators. A Supervisor can access and modify nodes in the whole simulation world.

5. What I Learned

By completing this work, I learned several important practical concepts.

First, I learned how a Webots world is structured. A simulation is not just a visual scene. It is a hierarchy of nodes, and each node has fields that control its position, size, appearance, physics, and behavior.

Second, I learned how to create physical objects. I now understand that if an object should interact with the world, it needs a `boundingObject` and a `Physics` node. Otherwise, it may only be visual or static.

Third, I learned how controllers work. A controller is a separate program that controls a robot through the Webots API. It must initialize the robot, get devices by name, enable sensors, run a feedback loop, and send commands to motors.

Fourth, I learned how distance sensors are used for obstacle avoidance. Sensor values must be interpreted correctly, and the robot's behavior depends on thresholds and motor speed decisions.

Fifth, I learned how to build a robot from scratch. This was the most important technical achievement. I created a robot body, added wheels, connected them with `HingeJoint` nodes, added motors, added sensors, and controlled everything using C code.

Sixth, I learned the purpose of PROTO files. A PROTO allows a robot design to become reusable and configurable. This makes the simulation cleaner and more professional.

Finally, I learned how Supervisor control works. A Supervisor can move objects, delete nodes, spawn robots or objects, track positions, and change fields during simulation.

6. Level I Reached

At the beginning of the work, my level in Webots was beginner. I did not fully understand the scene tree, nodes, robot controllers, or how objects interact with physics.

After completing the tutorials, I reached a basic-to-intermediate practical level. I can now:

- create a Webots project and world,
- add and modify objects,
- create simple physical objects,
- use DEF and USE,
- change appearance and materials,
- write basic C controllers,
- control motors,
- read distance sensors,
- create obstacle avoidance behavior,
- build a simple robot from scratch,
- create a reusable PROTO robot,
- and use Supervisor functions for scene control.

I do not consider myself advanced yet because I still need more practice with complex robot behavior, better navigation algorithms, advanced physics tuning, and larger simulation design. However, I now understand the main structure of Webots and can continue learning from this level.

7. What I Want to Improve Next

The main thing I want to improve is robot behavior. The current obstacle avoidance controller is basic. It works in simple cases, but it can fail in corners or tight spaces. I want to improve it by adding better recovery logic, such as reversing, turning at different angles, and detecting when the robot is stuck.

I also want to learn more about sensors. So far, I used distance sensors, but I want to understand cameras, GPS, lidar, inertial units, and how they can be used in more realistic robot applications.

Another area I want to improve is path planning. Instead of only reacting to obstacles, I want the robot to move toward a goal, avoid obstacles intelligently, and plan better paths inside the environment.

I also want to improve my understanding of physics settings. The physics tutorial showed that mass, friction, bounding objects, and contact properties can change the behavior a lot. I want to practice this more so I can create more stable and realistic simulations.

Finally, I want to become more comfortable with PROTO files and Supervisor controllers. These are useful for building reusable robots and controlling experiments automatically. I think learning these better would help me create more professional simulations.

8. Personal Reflection

This work was useful because it forced me to learn by doing. Reading the tutorial alone was not enough. I had to test the simulation, make mistakes, fix the controller, reset the world, adjust nodes, and try again.

The biggest lesson was that robot simulation is a combination of several parts. The world design, physical object setup, robot structure, device names, sensor values, and controller code all need to match. If one part is wrong, the robot may not work correctly.

The most difficult part was the four-wheeled robot and obstacle avoidance controller. The most interesting part was the Supervisor because it showed how the simulation can be controlled at a higher level.

Overall, I feel that I now understand the foundation of Webots. I can create simple simulations, program basic robot behavior, and explain the main parts of the system. My next goal is to move from basic tutorial-based work to more independent robot behavior, where I design the environment, robot, and controller myself.

9. Conclusion

This Webots practical work helped me develop a strong foundation in robot simulation. I started with basic world creation and gradually moved toward controllers, physics, robot modeling, PROTO files, and Supervisor control.

The work included several struggles, such as interface confusion, robot movement problems, obstacle avoidance limitations, physics behavior issues, and controller setup mistakes. These problems were useful because they helped me understand how Webots actually works, not only how to follow tutorial steps.

By the end of the work, I reached a level where I can create and modify Webots worlds, build simple physical objects, program robot controllers in C, use sensors and motors, build a basic four-wheeled robot, create a reusable PROTO, and use Supervisor functions.

For future improvement, I want to focus on better navigation, stronger obstacle avoidance, more sensors, improved physics configuration, and more advanced Supervisor-based simulations.

10. References

Cyberbotics. (2025). *Webots User Guide R2025a, Tutorial 1: Your First Simulation in Webots*.

Cyberbotics. (2025). *Webots User Guide R2025a, Tutorial 2: Modification of the Environment*.

Cyberbotics. (2025). *Webots User Guide R2025a, Tutorial 4: More about Controllers*.

Cyberbotics. (2025). *Webots User Guide R2025a, Tutorial 5: Compound Solid and Physics Attributes*.

Cyberbotics. (2025). *Webots User Guide R2025a, Tutorial 6: 4-Wheeled Robot*.

Cyberbotics. (2025). *Webots User Guide R2025a, Tutorial 8: The Supervisor*.